SSSM Backend

Spring Services Reference

Spring Boot 4.0.2 · Java 25 · Reactive WebFlux / R2DBC

11 services · Ports 8081 – 8097 · PostgreSQL + Cassandra + Elasticsearch

Table of Contents

user-service :8081 4
- Responsibilities 5
- Key DB Tables 5
- Kafka — Published 5
- gRPC Server — UserService 5
- API Endpoints 5
- Cache Keys 5

media-service :8082 6
- Responsibilities 6
- Upload Pipeline 6
- Kafka — Published 6
- API Endpoints 6

post-service :8083 7
- Responsibilities 7
- Post Types & Status 7
- Feed Algorithms 7
- Kafka — Published 7
- API Endpoints 7

comment-service :8084 9
- Responsibilities 9
- Why Cassandra 9
- Depth Model 9
- Sort Modes 9
- Kafka — Published 9
- API Endpoints 9

notification-service :8085 11
- Responsibilities 11
- Notification Types 11
- Cross-pod WS Routing 11
- Kafka — Consumed 11
- API Endpoints 11

search-service :8086 13
- Responsibilities 13
- Indices 13
- API Endpoints 13

group-service :8087 14
- Responsibilities 14

Group Visibility Modes 14
Role Hierarchy 14
Kafka — Published 14
private-message-service :8088 15
Responsibilities 15
Conversation Types 15
Real-time Architecture 15
Kafka — Published 15
message-notification-service :8089 16
Responsibilities 16
Push Decision Flow 16
Kafka — Consumed 16
API Endpoints 16
post-interaction-service :8096 17
Responsibilities 17
Why Cassandra 17
Bucket Display Strategy 17
View Count — Time-Buffered Flush 17
Kafka — Published 17
Kafka — Consumed 18
gRPC Server — PostInteractionService 18
API Endpoints 18
Rate Limits 18
bookmark-service :8097 19
Responsibilities 19
Denormalisation Strategy 19
Collection Lifecycle 19
Query Capabilities 19
Kafka — Published 19
Kafka — Consumed 19
API Endpoints 20
gRPC Server — BookmarkService 20
Rate Limits 20

Overview

This document covers all 11 Spring Boot microservices in the SSSM backend. Two new services were added in the latest release (marked ✦): post-interaction-service and bookmark-service, which split interaction counting and bookmark management out of post-service to scale those write-heavy workloads independently on Cassandra and PostgreSQL respectively.

Service	Port	Primary DB	Key responsibility
user-service	8081	PostgreSQL	Profiles, social graph, Keycloak sync
media-service	8082	PostgreSQL + S3	Upload, transcode, AWS CDN pipeline
post-service	8083	PostgreSQL	Post lifecycle, home & explore feeds
comment-service	8084	Cassandra	Nested comments (depth ≤ 3), reactions, view count
notification-service	8085	Cassandra	In-app push, real-time WebSocket delivery
search-service	8086	Elasticsearch	Full-text search, autocomplete, trending
group-service	8087	PostgreSQL	Groups, membership, roles, join screening
private-message-service	8088	Cassandra	DM, group chat, real-time WS delivery
message-notification-service	8089	Cassandra	FCM/APNs/Web Push for messages
post-interaction-service ✦	8096	Cassandra	Likes, shares, view tracking, bucket counters
bookmark-service ✦	8097	PostgreSQL	Bookmark CRUD, collections, FTS search

✦ = new service added in latest release.

user-service :8081

Port	8081
Primary DB	PostgreSQL (R2DBC) — schema sssm_users
Cache	Redis
Starters	postgres-starter · redis-starter · kafka-starter · metrics-starter · security-starter

Responsibilities

User identity, profiles, social graph (follow/unfollow), account settings, verification workflow, and Keycloak synchronisation. The most-called gRPC server in the system — post, comment, notification, and private-message services all query it.

Key DB Tables

users · followers · follow_requests · account_history · verifications — No foreign keys; referential integrity enforced at the application layer.

Kafka — Published

Topic	Action / Consumer
`user.profile.updated`	search-svc, user-analysis-svc
`user.avatar.changed`	post-svc (auto-post), search-svc
`user.followed`	notification-svc, user-analysis-svc
`user.unfollowed`	notification-svc
`user.verified`	notification-svc

gRPC Server — UserService

Exposes: findUserById · findUsersByIds · checkUserBlocked · getUserSettings · getNotifPreferences

API Endpoints

Method + Path	Description
`GET /api/v1/users/{username}`	Public profile
`GET /api/v1/users/me`	Own profile
`PUT /api/v1/users/me`	Update profile
`PUT /api/v1/users/me/avatar`	Change avatar → calls media-service
`POST /api/v1/users/{userId}/follow`	Follow user
`DELETE /api/v1/users/{userId}/follow`	Unfollow user
`GET /api/v1/users/me/follow-requests`	Pending follow requests
`PUT /api/v1/users/me/follow-requests/{reqId}`	Approve/reject request

Cache Keys

user:profile:{userId} · user:profile:username:{username} (5 min) · user:is-following:{a}:{b} (5 min) · user:settings:{userId} (10 min)

media-service :8082

Port	8082
Primary DB	PostgreSQL (R2DBC) — schema sssm_media
Cache	Redis
Starters	postgres-starter · redis-starter · kafka-starter · metrics-starter · security-starter · grpc-starter
Object Storage	AWS S3
CDN	AWS CloudFront

Responsibilities

Centralised media management. Accepts uploads, validates via Apache Tika magic-byte detection, compresses/transcodes with libvips & FFmpeg, delegates safety scan to media-guard-service, stores processed files on AWS S3, and serves via CloudFront CDN.

Upload Pipeline

```
Client  →  POST /api/v1/media/upload  (multipart)
          ↓
  1. Tika: detect MIME type (reject spoofed extensions)
  2. media-guard-service: NSFW + malware scan  (gRPC)
  3. libvips / FFmpeg: compress / transcode
  4. S3 PUT  →  CDN URL returned
  5. Kafka: media.upload.completed → post-service
```

Kafka — Published

Topic	Action / Consumer
`media.upload.completed`	post-svc (promotes PENDING_MEDIA → PUBLISHED)
`media.upload.failed`	post-svc (marks MEDIA_FAILED)

API Endpoints

Method + Path	Description
`POST /api/v1/media/upload`	Upload media (image/video/audio/document)
`GET /api/v1/media/{mediaId}`	Media metadata
`DELETE /api/v1/media/{mediaId}`	Soft-delete media asset

post-service :8083

Port	8083
Primary DB	PostgreSQL (R2DBC) — schema sssm_posts
Cache	Redis
Starters	postgres-starter · redis-starter · kafka-starter · metrics-starter · security-starter

Responsibilities

Core posting engine. Post lifecycle (CRUD, status transitions), home/explore feeds, group-context posts, and system auto-posts. Does NOT own interaction logic — likes, shares, bookmark counts, and view counters belong to post-interaction-service.

Post Types & Status

post_type	status values
ORIGINAL · REPOST · QUOTE · AUTO	DRAFT · PENDING_MEDIA · PUBLISHED · PENDING_REVIEW · HIDDEN · DELETED

Feed Algorithms

Home feed: posts from followed users, created_at DESC, cursor-paginated. Redis Sorted Set feed:home:{userId} TTL 30 s.

Explore feed: scored via BatchGetPostCounts gRPC from post-interaction-service: score = (like × 2 + share × 3 + comments) / (hours_since_posted + 2). Cache: feed:explore:global TTL 1 min.

Kafka — Published

Topic	Action / Consumer
`post.created`	notification-svc, search-svc, user-analysis-svc, post-interaction-svc
`post.updated`	search-svc
`post.deleted`	search-svc, comment-svc, post-interaction-svc, bookmark-svc
`post.reported`	ai-dashboard-svc

API Endpoints

Method + Path	Description
`POST /api/v1/posts`	Create post
`GET /api/v1/posts/{postId}`	Get post
`PUT /api/v1/posts/{postId}`	Edit post

`DELETE /api/v1/posts/{postId}`	Soft-delete
`POST /api/v1/posts/{postId}/repost`	Repost (creates new post with own id)
`GET /api/v1/posts/feed/home`	Home feed (cursor)
`GET /api/v1/posts/feed/explore`	Explore feed

comment-service :8084

Port	8084
Primary DB	Apache Cassandra — keyspace sssm_comments
Cache	Redis
Starters	cassandra-starter · redis-starter · kafka-starter · metrics-starter · security-starter

Responsibilities

High-throughput nested comment management (max depth = 3), comment reactions (6 types), view count via time-buffered flush, and comment ranking orchestration via gRPC to comment-recommendation-service for BEST sort.

Why Cassandra

Append-heavy writes (millions/day). TIMEUUID clustering key enables natural cursor pagination without OFFSET. COUNTER type provides lock-free atomic increments for reaction and reply counts. Simple query patterns by post_id and parent_comment_id.

Depth Model

```
depth = 0  →  Top-level comment
depth = 1  →  Reply to depth-0
depth = 2  →  Reply to depth-1
depth = 3  →  Reply to depth-2  ← MAX DEPTH

Reply attempt at depth-3  →  422 MAX REPLY DEPTH EXCEEDED
```

Sort Modes

Sort	Mechanism
NEWEST / OLDEST	Direct Cassandra cursor on top_comments_by_post
BEST	Fetch 3× candidates → gRPC RankComments → return top N in ranked order

Kafka — Published

Topic	Action / Consumer
`comment.reacted`	notification-svc (push to author)
`comment.view.dwell`	comment-recommendation-svc (dwell-time signal)
`comment.spam.detected`	Internal: sets status = SPAM_HIDDEN

API Endpoints

Method + Path	Description
`GET /api/v1/posts/{postId}/comments`	Paginated comments (sort=NEWEST|OLDEST|BEST)
`POST /api/v1/posts/{postId}/comments`	Add top-level comment
`GET /api/v1/comments/{commentId}/replies`	Paginated replies
`POST /api/v1/comments/{commentId}/replies`	Add reply
`POST /api/v1/comments/{commentId}/react`	React (LIKE|LOVE|HAHA|WOW|SAD|ANGRY)
`POST /api/v1/comments/{commentId}/view`	Register view with dwell_ms

notification-service :8085

Port	8085
Primary DB	Apache Cassandra — keyspace sssm_notifications
Cache	Redis
Starters	cassandra-starter · redis-starter · kafka-starter · metrics-starter · security-starter · websocket-starter

Responsibilities

Fan-out in-app notifications to users. Real-time delivery via reactive WebSocket. Multi-device read-state sync via Redis Pub/Sub across pods. Notification TTL is 90 days.

Notification Types

LIKE · COMMENT · REPLY · FOLLOW · FOLLOW_REQUEST · MENTION · REPOST · GROUP_INVITE · GROUP_JOIN · GROUP_ROLE_CHANGED · POST_APPROVED

Cross-pod WS Routing

```
Producer pod  →  Redis PUBLISH  notif:{userId}:{deviceId}
All pods subscribe → deliver via local WS session if present
device_sessions table tracks pod_id per device
```

Kafka — Consumed

Topic	Action / Consumer
`post.interaction.created`	Push like/share notification to post author
`comment.reacted`	Push reaction notification to comment author
`user.followed`	Push follow notification
`group.member.joined`	Push group join notification
`group.post.pinned`	Push pin notification to group members

API Endpoints

Method + Path	Description
`GET /api/v1/notifications`	List notifications (cursor-paginated)
`POST /api/v1/notifications/{id}/read`	Mark single notification read
`POST /api/v1/notifications/read-all`	Mark all read
`DELETE /api/v1/notifications/{id}`	Soft-delete notification

`GET /ws/notifications`	WebSocket endpoint

search-service :8086

Port	8086
Primary DB	Elasticsearch 8 — dedicated cluster
Cache	Redis
Starters	elasticsearch-starter · redis-starter · kafka-starter · metrics-starter · security-starter

Responsibilities

Full-text search across posts, users, hashtags, and groups. Autocomplete suggestions. Trending hashtags. Maintains denormalised Elasticsearch indices fed by Kafka CDC events. No PostgreSQL or Cassandra schema — owns Elasticsearch only.

Indices

Index	Refresh	Fields
`users_v1`	1 s	username, displayName, bio, isVerified
`posts_v1`	1 s	content, hashtags, mentions, groupId
`hashtags_v1`	10 s	tag name, trending score
`groups_v1`	5 s	name, description, tags, category

API Endpoints

Method + Path	Description
`GET /api/v1/search?q=&type=POST|USER|HASHTAG|GROUP`	Unified search with type filter
`GET /api/v1/search/autocomplete?q=&type=`	Autocomplete suggestions
`GET /api/v1/search/trending-hashtags`	Trending hashtags list

group-service :8087

Port	8087
Primary DB	PostgreSQL (R2DBC) — schema sssm_groups
Cache	Redis
Starters	postgres-starter · redis-starter · kafka-starter · metrics-starter · security-starter

Responsibilities

Social group management: creation, membership lifecycle, role hierarchy, join policies, anti-bot screening via join questions, content pinning (max 5 per group), tags, and group settings. Posts inside a group are created via post-service; group-service owns the group context only.

Group Visibility Modes

Mode	Discoverable	Join mechanism
PUBLIC	Yes	Instant (or queue if autoApprove=false)
PRIVATE	Yes	Request + optional screening questions
INVITE_ONLY	No	Invitation only

Role Hierarchy

OWNER > ADMIN > MODERATOR > MEMBER. Owners can delete the group and transfer ownership. Admins promote/demote moderators. Moderators approve join requests, pin posts, and ban members.

Kafka — Published

Topic	Action / Consumer
`group.created / group.updated / group.deleted`	search-svc, user-analysis-svc
`group.member.joined / group.member.left`	notification-svc, private-message-svc
`group.member.role.changed / group.member.banned`	notification-svc
`group.post.created`	search-svc

private-message-service :8088

Port	8088
Primary DB	Apache Cassandra — keyspace sssm_messages
Cache	Redis (metadata cache + Pub/Sub for cross-pod WS routing)
Starters	cassandra-starter · redis-starter · kafka-starter · metrics-starter · security-starter · websocket-starter · grpc-starter

Responsibilities

End-to-end private messaging: 1-on-1 DMs, user-created group chats (up to 500 members), and group-channel conversations linked to social groups. Handles real-time WebSocket delivery, reactions, forwarding, read receipts, typing indicators, per-conversation notification settings, and media attachments.

Conversation Types

Type	Max members	Created by
DIRECT	2	Either user
GROUP_CHAT	500	Any user
GROUP_CHANNEL	Unlimited*	group-service (auto-created)

Real-time Architecture

```
Sender (WS pod-1)
  │ send message
  ▼
private-message-service (pod-1):
  1. Write to Cassandra
  2. PUBLISH Redis channel  conv:{convId}   ← cross-pod routing
  3. Publish Kafka: message.sent             ← offline push

Redis Pub/Sub fanout:
  pod-1 → deliver to sender (confirmation)
  pod-N → deliver to online recipients

Kafka message.sent → message-notification-service → FCM / APNs / Web Push
```

Kafka — Published

Topic	Action / Consumer
`message.sent`	message-notification-svc (offline push)
`conversation.created`	user-analysis-svc

message-notification-service :8089

Port	8089
Primary DB	Apache Cassandra — keyspace sssm_msg_notifications
Cache	Redis
Starters	cassandra-starter · redis-starter · kafka-starter · metrics-starter · security-starter
Push channels	FCM · APNs · Web Push (VAPID)

Responsibilities

Push notification delivery for private messages. Separated from notification-service because message notifications require multi-channel push (FCM/APNs/Web Push), message-preview masking, sliding-window batching to prevent spamming, allowDmFrom filtering, and device-token lifecycle management.

Push Decision Flow

```
Kafka: message.sent
  ↓
  1. allowDmFrom filter (user settings gRPC → user-svc)
  2. Online check (Redis: user:{userId}:online)
     → ONLINE  → skip push (WS delivers)
     → OFFLINE → determine platform from device_push_tokens
  3. Sliding window: ≤ 5 pushes/min per conv → batch remaining
  4. Send FCM / APNs / VAPID Web Push
  5. Log to message_notification_log (TTL 30 days)
```

Kafka — Consumed

Topic	Action / Consumer
`message.sent`	Trigger push notification decision flow
`user.settings.updated`	Invalidate allowDmFrom cache

API Endpoints

Method + Path	Description
`POST /api/v1/msg-notifications/devices`	Register push token
`DELETE /api/v1/msg-notifications/devices/{deviceId}`	Unregister token
`PUT /api/v1/msg-notifications/settings`	Update DM notification preferences

post-interaction-service :8096

Port	8096
Primary DB	Apache Cassandra — keyspace sssm_post_interactions
Cache	Redis
Starters	cassandra-starter · redis-starter · kafka-starter · metrics-starter · security-starter

Responsibilities

Owns all post-level interaction signals: like, share (repost), and bookmark counting. Maintains interaction counters with bucket-display strategy, stores actor lists for author lookup (private like X), manages view count via time-buffered flush, and emits Kafka events for like/share notifications. Bookmark CRUD belongs to bookmark-service.

Why Cassandra

Write-heavy workload (millions of likes/views per day). COUNTER columns provide atomic lock-free increments. Partition by (post_id, interaction_type) gives O(1) actor-list lookup even for viral posts. No JOIN or distributed transaction needed.

Bucket Display Strategy

Raw count	Display	Bucket unit
0 – 999	Exact (e.g. 999)	1
1 000 – 9 999	1.2K	100
10 000 – 99 999	12K	1 000
100 000 – 999 999	123K	10 000
1 000 000+	1.2M	100 000

Redis cache post:display-count:{postId} is only updated when the display string crosses a bucket boundary — eliminating constant cache invalidation on high-traffic posts.

View Count — Time-Buffered Flush

```
POST /posts/{postId}/view
  → Redis INCR  post:view-buffer:{postId}  (TTL 120 s rolling)

ViewFlushJob every 60 s:
  SCAN  post:view-buffer:*
  GETDEL key  → delta
  Cassandra COUNTER += delta  (single batch write)
  Recalculate display string if bucket changed
```

Kafka — Published

Topic	Action / Consumer
`post.interaction.created`	notification-svc (like/share notification), user-analysis-svc
`post.interaction.deleted`	user-analysis-svc

Kafka — Consumed

Topic	Action / Consumer
`post.bookmarked`	Increment bookmark_count + write actor record
`post.unbookmarked`	Decrement bookmark_count + remove actor record
`post.deleted`	Soft-clean counters (retain history 30 days)

gRPC Server — PostInteractionService

GetPostCounts · GetUserInteraction · BatchGetPostCounts — consumed by post-service for feed scoring and by bookmark-service to verify interaction state.

API Endpoints

Method + Path	Description
`POST /api/v1/posts/{postId}/like`	Like a post
`DELETE /api/v1/posts/{postId}/like`	Unlike
`POST /api/v1/posts/{postId}/share`	Register share signal
`POST /api/v1/posts/{postId}/view`	Register view
`GET /api/v1/posts/{postId}/counts`	Display counts (Redis-first)
`GET /api/v1/posts/{postId}/my-interactions`	Caller's like/share/bookmark state
`GET /api/v1/posts/{postId}/likers`	Author-only: who liked (cursor)

Rate Limits

Endpoint	Limit
POST /like	300 / hour per userId
POST /view	10 / min per userId per postId

bookmark-service :8097

Port	8097
Primary DB	PostgreSQL (R2DBC) — schema sssm_bookmarks
Cache	Redis
Starters	postgres-starter · redis-starter · kafka-starter · metrics-starter · security-starter

Responsibilities

Full bookmark CRUD: save posts into named collections, categorise by colour/icon, full-text search over previews, range filtering by save date or post date, sort options, and bulk delete. Does NOT own the bookmark counter on posts — it emits Kafka events for post-interaction-service to update counts.

Denormalisation Strategy

When saving a bookmark, the service calls gRPC to post-service to snapshot: author_id, content preview (≤ 280 chars), created_at, and first media thumbnail. This avoids cross-service joins at query time. If a post is later deleted, the post.deleted Kafka event updates the preview to '[Post removed]'.

Collection Lifecycle

```
Auto-create 'Saved' collection (is_default=TRUE) on first bookmark

Delete collection:
  is_default=TRUE  → 403 Forbidden
  has bookmarks    → 409 Conflict (move to default first)
  empty            → soft-delete + decrement post_count = 0

Save: INSERT bookmarks + UPDATE collection.post_count + Kafka post.bookmarked
Unsave: DELETE bookmark + DECR post count + Kafka post.unbookmarked
```

Query Capabilities

Range filter by saved_at or post_created_at. Sort: SAVED_AT_DESC (default) | SAVED_AT_ASC | POST_DATE_DESC | POST_DATE_ASC. Full-text search via PostgreSQL GIN index on post_content_preview. Bulk delete by postId array.

Kafka — Published

Topic	Action / Consumer
`post.bookmarked`	post-interaction-svc (increment counter)
`post.unbookmarked`	post-interaction-svc (decrement counter)

Kafka — Consumed

Topic	Action / Consumer
`post.deleted`	Update post_content_preview to '[Post removed]'

`user.deleted`	Hard-delete all bookmarks and collections

API Endpoints

Method + Path	Description
`GET /api/v1/bookmarks/collections`	List own collections
`POST /api/v1/bookmarks/collections`	Create collection
`POST /api/v1/bookmarks`	Save post (body: {postId, collectionId?, note?})
`DELETE /api/v1/bookmarks/{postId}`	Unsave post
`GET /api/v1/bookmarks?collectionId=&sort=&page=&size=`	List bookmarks (range / sort)
`GET /api/v1/bookmarks/search?q=&collectionId=`	Full-text search
`DELETE /api/v1/bookmarks/bulk`	Bulk delete by postId array
`GET /api/v1/bookmarks/check/{postId}`	Check if bookmarked

gRPC Server — BookmarkService

CheckBookmarked · BatchCheckBookmarked — consumed by post-service when building feed responses.

Rate Limits

Endpoint	Limit
POST /bookmarks	500 / day per userId
POST /collections	20 / day · max 50 collections per user